

# K8S-06: Namespace Organization and Boundaries

---

**Series:** K8S — Kubernetes Monitoring | **Notebook:** 6 of 13 | **Created:** January 2026  
| **Last Updated:** 09/24/2026

## Organizing Kubernetes Monitoring with Namespaces

---

Namespaces provide logical boundaries in Kubernetes for resource isolation, access control, and organizational structure. This notebook covers namespace strategies and how to leverage them in Dynatrace for filtered views, access control, and cost allocation.

---

## Table of Contents

---

1. [Namespace Strategies](#)
  2. [Namespace Monitoring in Dynatrace](#)
  3. [Resource Quotas and Limits](#)
  4. [Namespace-Based Access Control](#)
  5. [Multi-Tenant Clusters](#)
  6. [Cost Allocation by Namespace](#)
  7. [Namespace Best Practices](#)
-

# Prerequisites

| Requirement           | Details                                         |
|-----------------------|-------------------------------------------------|
| Dynatrace Environment | SaaS with Kubernetes monitoring                 |
| DynaKube              | Deployed with namespace selector configured     |
| Permissions           | entities.read , metrics.read                    |
| Knowledge             | K8S-01 Fundamentals, K8S-05 Workload Monitoring |

## 1. Namespace Strategies

### Common Namespace Patterns

| Strategy       | Structure                      | Use Case                      |
|----------------|--------------------------------|-------------------------------|
| By Environment | dev , staging , prod           | Single cluster, multiple envs |
| By Team        | team-checkout , team-catalog   | Team ownership                |
| By Application | app-frontend , app-backend     | Application isolation         |
| By Function    | monitoring , logging , ingress | Infrastructure services       |
| Hybrid         | team-checkout-prod             | Combined approach             |

### Namespace Naming Conventions

<prefix> - <owner> - <env>;  
  
Examples:  
app-checkout-prod  
team-platform-dev  
infra-monitoring

## System Namespaces

| Namespace       | Purpose             | Monitor?       |
|-----------------|---------------------|----------------|
| kube-system     | Core K8s components | Yes (critical) |
| kube-public     | Public resources    | Minimal        |
| kube-node-lease | Node heartbeats     | No             |
| default         | Catch-all           | Discourage use |

```
// List all Kubernetes namespaces (smartscape topology)
smartscapeNodes "K8S_NAMESPACE"
| fields entity.name = name, tags
| sort entity.name asc
| limit 50

// Legacy alternative (deprecated for new content):
// fetch dt.entity.cloud_application_namespace
// | fields entity.name, tags
// | sort entity.name asc
// | limit 50
```

## 2. Namespace Monitoring in Dynatrace

### DynaKube Namespace Selector

Control which namespaces receive OneAgent injection:

```
spec:
  cloudNativeFullStack: # namespaceSelector goes inside the deployment mode
    section
      namespaceSelector:
        matchLabels:
          monitoring: dynatrace
```

Or exclude specific namespaces:

```
spec:
  cloudNativeFullStack: # namespaceSelector goes inside the deployment mode
section
  namespaceSelector:
    matchExpressions:
      - key: monitoring
        operator: NotIn
        values:
          - disabled
```

## Namespace Labels for Filtering

| Label       | Purpose            | Example              |
|-------------|--------------------|----------------------|
| team        | Team ownership     | team: checkout       |
| env         | Environment        | env: production      |
| cost-center | Billing allocation | cost-center: eng-123 |
| monitoring  | Injection control  | monitoring: enabled  |

```
// Namespace CPU usage – sum() adds up every container in the namespace; avg()
would
// return the average container's usage, which ranks namespaces by container
size, not consumption
timeseries cpuMillicores = sum(dt.kubernetes.container.cpu_usage), from:-1h,
by:{k8s.namespace.name}
| fieldsAdd avgCpuMillicores = arrayAvg(cpuMillicores)
| sort avgCpuMillicores desc
| limit 15
```

```
// Memory usage by namespace – sum() of every container's working set in the
namespace
timeseries memBytes = sum(dt.kubernetes.container.memory_working_set),
from:-1h, by:{k8s.namespace.name}
| fieldsAdd avgMemBytes = arrayAvg(memBytes)
| sort avgMemBytes desc
| limit 15
```

## Per-Namespace Service Detection Scoping

Service detection settings — including **Enhanced Endpoints for SDv1** (Dynatrace v1.329+) — can be overridden per Kubernetes namespace, not just at the environment level. Useful when one namespace runs services that need different endpoint-naming behavior than the tenant default.

**Override path:** *Kubernetes app* → *select cluster or namespace* → *Actions menu* → *Service detection settings* → *Process and contextualize* → *Services* → *Service detection v1*

**Scoping precedence (most specific wins):**

| Scope                | When to use                                                                                                                                                                |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Environment-wide     | Default policy for the whole tenant                                                                                                                                        |
| Host group           | Group of hosts with shared service-detection needs                                                                                                                         |
| Kubernetes cluster   | All namespaces in one cluster (e.g., a multi-tenant cluster owned by one team)                                                                                             |
| Kubernetes namespace | One workload — typically when a service behind a reverse proxy (Nginx/Apache/IIS) collapses to <code>GET /*</code> and you want to keep the per-endpoint metrics elsewhere |

**Tenant-creation-date reminder:** environments created at v1.333+ have Enhanced Endpoints always on and not configurable at any scope. Per-namespace overrides apply only to older environments where the setting is toggleable.

Sources: [Enhanced endpoints for SDv1 \(DT docs\)](#).

## 3. Resource Quotas and Limits

### ResourceQuota Monitoring

ResourceQuotas limit total resource consumption per namespace:

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: team-checkout
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "20"
    limits.memory: 40Gi
    pods: "50"
```

## LimitRange for Defaults

```
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: team-checkout
spec:
  limits:
    - default:
        cpu: "500m"
        memory: "512Mi"
      defaultRequest:
        cpu: "100m"
        memory: "128Mi"
      type: Container
```

## Quota Utilization Metrics

| Metric             | Grail key                                                               | Alert When        |
|--------------------|-------------------------------------------------------------------------|-------------------|
| CPU quota usage    | <code>dt.kubernetes.container.requests_cpu</code> (sum by namespace)    | >80% of quota     |
| Memory quota usage | <code>dt.kubernetes.container.requests_memory</code> (sum by namespace) | >80% of quota     |
| Pod count          | <code>dt.kubernetes.pods</code>                                         | Approaching limit |

**Requests are container-grain.** A ResourceQuota is enforced per namespace, but Dynatrace emits requests per **container** — there is no

```
dt.kubernetes.workload.requests_cpu or .requests_memory . Sum the  
dt.kubernetes.container.* series across k8s.namespace.name to get the figure  
the quota is measured against. Confirm what your tenant carries with metrics |  
filter startsWith(metric.key, "dt.kubernetes") | summarize n = count(), by:  
{metric.key} — note metrics takes from: with no leading comma.
```

**Quota tracking across ActiveGate 1.343 / 1.345 upgrades:** request metrics carry an effective pod value — init containers are accounted for from ActiveGate 1.343, and pod-level `spec.resources` (Kubernetes 1.34+) and pod `overhead` from 1.345 — so reserved-CPU and reserved-memory figures can step at either upgrade with no workload change. Read a trend spanning either boundary as **two series, not one**, and re-baseline any >80%-of-quota alert afterward. Full explanation: K8S-08 §2.

```
// CPU requests by namespace (quota tracking)  
// Requests are emitted at container grain – there is no  
dt.kubernetes.workload.requests_cpu.  
// Sum across containers to get the namespace reservation a ResourceQuota is  
measured against.  
timeseries cpuReq = sum(dt.kubernetes.container.requests_cpu), from:-1h, by:  
{k8s.namespace.name}  
| fieldsAdd avgReqMillicores = round(arrayAvg(cpuReq), decimals: 0)  
| fields k8s.namespace.name, avgReqMillicores  
| sort avgReqMillicores desc  
| limit 15
```

```
// Memory requests by namespace (GiB, quota tracking)  
timeseries memReq = sum(dt.kubernetes.container.requests_memory), from:-1h,  
by:{k8s.namespace.name}  
| fieldsAdd avgReqGiB = round(arrayAvg(memReq) / 1073741824, decimals: 2)  
| fields k8s.namespace.name, avgReqGiB  
| sort avgReqGiB desc  
| limit 15
```

## 4. Namespace-Based Access Control

### Dynatrace IAM with Boundaries

Use Dynatrace boundaries to restrict visibility by namespace:

**Policy Example (Grant access to checkout namespace):**

```
{
  "name": "pol-namespace-checkout-viewer",
  "statement": {
    "effect": "ALLOW",
    "permissions": ["environment:roles:viewer"],
    "conditions": [
      {
        "kubernetes.namespace": "checkout"
      }
    ]
  }
}
```

### Segment by Namespace

Create segments for namespace-based filtering:

| Segment        | Filter                                         | Use Case      |
|----------------|------------------------------------------------|---------------|
| seg-checkout   | k8s.namespace.name == "checkout"               | Team view     |
| seg-production | k8s.namespace.name matches "*-prod"            | Prod only     |
| seg-platform   | k8s.namespace.name in ("monitoring","logging") | Platform team |

### Kubernetes RBAC Alignment

Align Dynatrace access with Kubernetes RBAC:

| K8s Role        | Dynatrace Access          |
|-----------------|---------------------------|
| namespace-admin | Full namespace visibility |

| K8s Role         | Dynatrace Access         |
|------------------|--------------------------|
| namespace-viewer | Read-only namespace data |
| cluster-admin    | All namespaces           |

## 5. Multi-Tenant Clusters

### Tenant Isolation Strategies

| Level   | Mechanism                 | Dynatrace Support    |
|---------|---------------------------|----------------------|
| Soft    | Namespace + NetworkPolicy | Boundaries, segments |
| Hard    | Separate clusters         | Separate DynaKubes   |
| Virtual | vCluster                  | Namespace labels     |

### DynaKube per Tenant (Hard Isolation)

For strong isolation, use separate DynaKube instances:

```
# Tenant A
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube-tenant-a
  namespace: dynatrace
spec:
  apiUrl: https://tenant-a.live.dynatrace.com/api
  namespaceSelector:
    matchLabels:
      tenant: a
```

### Shared DynaKube (Soft Isolation)

Single DynaKube with boundary-based access control:

```
spec:
  oneAgent:
    cloudNativeFullStack: {}
  # All namespaces monitored, access controlled via IAM
```

```
// Workload count via smartscape topology
smartscapeNodes "K8S_DEPLOYMENT"
| summarize workloadCount = count()
| limit 20

// For StatefulSets / DaemonSets / Pods, substitute the type:
// smartscapeNodes "K8S_STATEFULSET"
// smartscapeNodes "K8S_DAEMONSET"
// smartscapeNodes "K8S_POD"

// Legacy alternative (deprecated for new content):
// fetch dt.entity.cloud_application
// | summarize workloadCount = count()
// | limit 20
```

## 6. Cost Allocation by Namespace

### Resource-Based Cost Allocation

Calculate costs based on resource consumption:

| Resource | Cost Metric | Calculation                 |
|----------|-------------|-----------------------------|
| CPU      | Core-hours  | CPU usage × hours × rate    |
| Memory   | GB-hours    | Memory usage × hours × rate |
| Storage  | GB-hours    | PVC size × hours × rate     |

# Labels for Cost Centers

```
apiVersion: v1
kind: Namespace
metadata:
  name: team-checkout
  labels:
    cost-center: "eng-checkout-123"
    team: "checkout"
    budget-owner: "alice@example.com"
```

```
// Resource consumption by namespace (for cost allocation) – sum() of every
// container in the
// namespace; a per-container avg() under-states namespaces that run many
// modest containers
timeseries cpuMillicores = sum(dt.kubernetes.container.cpu_usage), from:-1h,
by:{k8s.namespace.name}
| fieldsAdd avgCpuMillicores = arrayAvg(cpuMillicores)
| sort avgCpuMillicores desc
| limit 15
```

## 7. Namespace Best Practices

### Naming and Organization

| Practice                             | Reason                           |
|--------------------------------------|----------------------------------|
| Use consistent naming                | Easier filtering and automation  |
| Apply standard labels                | Cost allocation, access control  |
| Avoid <code>default</code> namespace | Encourages explicit organization |
| Document ownership                   | Clear responsibility             |

### Resource Management

| Practice           | Reason                   |
|--------------------|--------------------------|
| Set ResourceQuotas | Prevent resource hogging |

| Practice              | Reason                      |
|-----------------------|-----------------------------|
| Configure LimitRanges | Ensure defaults             |
| Monitor quota usage   | Proactive capacity planning |

## Security

| Practice                     | Reason                  |
|------------------------------|-------------------------|
| Apply NetworkPolicies        | Namespace isolation     |
| Use RBAC per namespace       | Least privilege         |
| Separate sensitive workloads | Compliance requirements |

## Monitoring

| Practice                          | Reason                 |
|-----------------------------------|------------------------|
| Label namespaces for filtering    | Easy DQL queries       |
| Create namespace-based dashboards | Team-specific views    |
| Set up namespace alerts           | Targeted notifications |

## Next Steps

With namespace organization in place, proceed to:

| Next Notebook                     | Topic                     |
|-----------------------------------|---------------------------|
| <b>K8S-07: Events and Logs</b>    | Kubernetes event analysis |
| <b>K8S-08: DQL for Kubernetes</b> | Advanced query patterns   |
| <b>K8S-09: Troubleshooting</b>    | Debugging K8s monitoring  |

# Summary

---

In this notebook, you learned:

- Namespace strategies and naming conventions
  - DynaKube namespace selector configuration
  - Resource quota and limit monitoring
  - Namespace-based access control with boundaries
  - Multi-tenant cluster patterns
  - Cost allocation by namespace
  - Namespace best practices
- 

# References

---

- [Set up Dynatrace on Kubernetes \(DT docs\)](#)
  - [Kubernetes app — namespaces view \(DT docs\)](#)
  - [DynaKube parameters \(DT docs\)](#)
  - [Manage user permissions / boundaries \(DT docs\)](#)
  - [Kubernetes Namespaces \(kubernetes.io\)](#)
  - [Resource Quotas \(kubernetes.io\)](#)
  - [smartscapeNodes command \(DT docs\)](#)
  - [Effective pod resources \(DT docs\)](#)
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*